

Seoul Node Runner — Lightning Pool Browser

Design Specification for Claude Code on Umbrel

Project: AI-powered Lightning Network Liquidity Pool Browser **Node:** Seoul Node Runner
(028a108449df4d5f1f641cf08586f4b21fe66bc93b19a604da49a2811361cf0360) **Stack:** Python
backend + HTML/JS frontend, runs locally on Umbrel **Telegram:**
@Seoul_Node_Runner_bot

Overview

A self-hosted web dashboard that pulls the LightningNetwork.Plus (LN+) Liquidity Pool in real time, scores every node against Seoul Node Runner's current state, and presents a clean sortable table with AI recommendations, hover cards, and persistent bookmarks.

The goal is to replace manual browsing of LN+ with an intelligent local tool that tells the operator exactly who to do business with and why — and lets them save candidates for follow-up.

Core Features

1. Pool Node Table

Pull all nodes from the LN+ Liquidity Pool API and display as a sortable, filterable table with these columns:

Column	Source	Notes
Bookmark	Local storage	Star toggle, persists across sessions
Node alias	LN+ API	With avatar initials
Capacity	LN+ API	In BTC
Channels	LN+ API	Public channel count
Happy %	LN+ API	With total ratings count
Min channel	LN+ API	Must be $\leq$ Seoul Node Runner's max (2M sat)
Rank	LN+ API	e.g. "7 / Silver"
Connection	LN+ API	Cleartnet / Tor / Both
Last seen	LN+ / Amboss	Recency flag
AI score	Local scoring	Green / Amber / Red badge

2. AI Scoring Engine

Score each node automatically against Seoul Node Runner's current state:

Green (Recommended):

- Last seen < 24 hours
- Happy % $\geq$ 80%
- Ratings $\geq$ 5
- No Admin ban / Inactive tag
- Public channels $\geq$ 10
- Min channel size $\leq$ 2,000,000 sat (Seoul Node Runner max)
- Connection: Cleartnet + Tor preferred
- Not already a peer of Seoul Node Runner

Amber (Watch):

- Last seen 1-7 days
- Happy % 60-79%
- Ratings 2-4
- Tor only connection

- Low channel count (3-9)
- New node with no rating history

Red (Pass):

- Last seen > 7 days
- Happy % < 60%
- Admin ban or Inactive tag
- 0 channels or 0 SAT capacity
- Already connected to Seoul Node Runner (duplicate)

Each node gets a one-line AI reason string explaining the score.

3. Hover Card

On mouse hover over any table row, show a popup card with:

- Node alias + Prime Node badge if applicable
- Last seen
- Connection type
- Happy % and total ratings
- LN+ credits available
- Hardware notes (if in description)
- AI score reason

4. Bookmark System

- Star icon on each row — click to toggle bookmark
- Bookmarks persist to local storage (survives page refresh)
- Dedicated **Bookmarks tab** shows full node cards for saved nodes
- Each bookmark card includes:
 - Full node stats grid
 - AI score and reason
 - Action buttons: Full Analysis, Draft Message, Lookup on Amboss

5. Bookmarks Tab — Action Buttons

Each bookmarked node card has three quick actions:

Full Analysis — sends node data to the AI for a deep evaluation against Seoul Node Runner's routing strategy

Draft Message — generates a professional outreach message to send to the node operator via Amboss or LN+ DM, using Seoul Node Runner's established template:

```
"Hi [alias] — Seoul Node Runner here, a dedicated routing node near Seoul, South Korea. Running LND on dedicated hardware, dual-stack clearnet + Tor, LNDg AR automated rebalancing 24/7. [context-specific line]. —Seoul Node Runner (@tomcatt7)"
```

Lookup — opens the node's Amboss page and LN+ profile in new tabs

Data Sources

LN+ API (Primary)

```
Base URL: https://lightningnetwork.plus/api/2/
```

Key endpoints:

```
GET /get_swaps          - open liquidity swaps
GET /get_node/pubkey=    - individual node stats
GET /pool_nodes          - liquidity pool node list
```

No authentication required for public endpoints.

Amboss API (Secondary — node enrichment)

```
URL: https://api.amboss.space/graphql
Method: POST (GraphQL)
Auth: Bearer token in Authorization header
Key: Seoul-Node-Runner-Agent (stored in .env, expires June 2027)
```

Use for: verification status, channel graph data, last seen timestamp, social links.

LND REST API (Local — Seoul Node Runner state)

```
Base URL: https://umbrel.local:8080
Auth: macaroon header (read-only macaroon)
```

Pull on startup:

- Current peers (to flag duplicate connections)
- Channel list (local/remote balances)

- On-chain balance (to check if enough sats to open)
- Node pubkey (for self-exclusion from pool list)

Local Mempool API (Fee gate)

```
Primary:  http://umbrel.local:3000/api/v1/fees/recommended
Fallback: https://mempool.space/api/v1/fees/recommended
```

Always use local Umbrel mempool first — faster, private, no external dependency.

Response:

```
json
{
  "fastestFee": 1,
  "halfHourFee": 1,
  "hourFee": 1,
  "economyFee": 1,
  "minimumFee": 1
}
```

Block any on-chain action if `economyFee` > 5 sat/vB (configurable threshold).

Seoul Node Runner Current State

The scoring engine needs these values — pull from LND REST on startup:

```
python
SEOUL_NODE = {
    "pubkey": "028a108449df4d5f1f641cf08586f4b21fe66bc93b19a604da49a2811361cf03",
    "alias": "Seoul Node Runner",
    "min_channel_sat": 500_000,
    "max_channel_sat": 2_000_000,
    "preferred_connection": ["clearnet", "tor"],
    "onchain_balance_sat": None, # fetch from LND
    "current_peers": [],        # fetch from LND
    "telegram": "@tomcatt7"
}
```

Peer Vetting Criteria

These red flags auto-score a node Red regardless of other factors:

- banned_by_admin: true
- inactive: true
- last_seen > 7 days
- public_channels == 0
- capacity_sat == 0
- happy_pct < 50
- Already connected to Seoul Node Runner (pubkey in current_peers)
- Min channel size > Seoul Node Runner max channel (2,000,000 sat)

UI Layout

⚡ Seoul Node Runner – Pool Browser

[Pool Nodes] [Bookmarks ★3]

Stats: 3 Recommended | 2 Watch | 2 Pass

[Search...] [Filter ▼] [AI Suggest ↗]

★	Node	Cap	Ch	Rating	Score
★	SpookyPayment...	1.48BTC	✓	Rec	
★	LNRouteMaster	2.1 BTC	✓	Rec	
★	TorOnlyNode42	0.31BTC	~	Watch	
★	GhostNode_LN	0.01BTC	×	Pass	

Bookmarks Page Layout

Bookmarks (3 nodes saved) [Compare ↗]

SP SpookyPaymentAtADistance ★

✓ Recommended			
Capacity	Channels	Happy	
1.483 BTC	39	100% (80)	
Min ch.	Rank	Last seen	
1.0M	7/Silver	2h ago	
AI: 80 ratings, Prime, dual-stack			
[Full Analysis ↗][Draft Msg ↗][Lookup ↗]			

File Structure

```
~/seoul-node-runner/
├── .env                # API keys, never committed
├── config.py           # Seoul Node Runner settings
├── main.py             # Flask/FastAPI server
├── scorer.py           # AI scoring engine
├── data/
│   ├── ln_plus.py      # LN+ API client
│   ├── amboss.py       # Amboss GraphQL client
│   ├── lnd.py          # LND REST client
│   └── mempool.py       # Local mempool client
├── static/
│   ├── index.html      # Pool browser UI
│   ├── bookmarks.html  # Bookmarks page (or tab)
│   └── style.css
└── storage/
    └── bookmarks.json   # Persistent bookmarks
```

.env Template

```
bash
```

```
# Amboss API – expires June 2027
AMBOSS_API_KEY=your-key-here

# LND macaroon (read-only, base64)
LND_MACAROON=your-macaroon-here
LND_REST_URL=https://umbrel.local:8080

# Mempool
MEMPOOL_LOCAL=http://umbrel.local:3000/api/v1/fees/recommended
MEMPOOL_FALLBACK=https://mempool.space/api/v1/fees/recommended
MEMPOOL_FEE_THRESHOLD=5

# Telegram
TELEGRAM_BOT_TOKEN=your-token-here
TELEGRAM_CHAT_ID=your-chat-id-here

# LN+ (no auth needed for public endpoints)
LN_PLUS_BASE=https://lightningnetwork.plus/api/2
```

Telegram Integration

When the pool browser finds a new Recommended node that wasn't there on the last scan, push a Telegram alert:

```
⚡ New Pool Candidate – Seoul Node Runner

Node: SpookyPaymentAtADistance
Score: ✓ Recommended
Capacity: 1.483 BTC | Channels: 39
Happy: 100% (80 ratings) | Prime Node
Min channel: 1.0M sat | Rank: 7 / Silver
Last seen: 2h ago | Clearnet + Tor

Reason: 80 ratings, 100% happy, Prime node,
UPS+RAID hardware, dual-stack

Open browser: http://umbrel.local:5000
```


Refresh Schedule

Data	Frequency	Notes
Pool nodes	Every 6 hours	Full refresh
Node scores	On each refresh	Re-score against current state
Seoul Node Runner state	On startup + every hour	Peers, balances
Mempool fees	Before any action	Always fresh
Telegram alerts	On new Recommended nodes	Deduped — no repeat alerts

Phase 1 Deliverable (MVP)

Build first:

1. Flask server with `/api/pool` endpoint returning scored node list
2. Single HTML page with sortable table + AI score badges
3. Hover cards with node stats
4. Bookmark toggle with localStorage persistence
5. Bookmarks tab with full cards and action buttons
6. Static mock data acceptable for initial build — swap for live API in Phase 2

Phase 2

1. Wire in live LN+ Pool API
2. Wire in Amboss GraphQL for node enrichment
3. Wire in LND REST for peer deduplication
4. Telegram new-candidate alerts
5. Mempool fee gate

Phase 3

1. Add Triangle Scout tab (LN+ swap scouting)
2. Add Channel Health tab (LNDg integration)
3. Add Fee Manager tab (auto-fee adjustment)
4. Full Seoul Node Runner AI Agent dashboard

Notes for Claude Code

- Run on Umbrel at `http://umbrel.local:5000` — no external access needed
- All API keys in `.env` — never hardcode
- Use local mempool first, always
- Seoul Node Runner pubkey must be excluded from pool results (no self-connection)
- Bookmarks must persist — use `storage/bookmarks.json` as fallback if localStorage unavailable
- Dark mode support required — Umbrel uses dark theme
- Mobile-friendly layout preferred — sometimes accessed from phone
- No external CDN dependencies in production — bundle everything locally

Generated by Claude (CC) — Session: Running LND on Umbrel Part 2 Seoul Node Runner / Pyeongtaek-si, South Korea / June 2026